

Semillitas

Lider: Victor Hugo Grajeda Cortes

Cesar Damian Huerta Vera

Efrain Rosales Rocha

Asesor: Julio Esteban Valdes Lopez

CENTRO UNIVERSITARIO DE CIENCIAS

EXACTAS E INGENIERÍAS, (CUCEI, UDG)

victor.grajeda5489@alumnos.udg.mx

✉

cesar.huerta4469@alumnos.udg.mx

efrain.rosales@alumnos.udg.mx

julio.valdes@academicos.udg.mx

Abstract– Tras la pandemia, México presenta un rezago crítico en lectoescritura y matemáticas en el 80% de niños de 10 años, lo que motiva el desarrollo de Semillitas, una aplicación móvil diseñada para aminorar esta tendencia mediante apoyo pedagógico temprano. Para su ejecución se empleó una metodología ágil basada en Sprints e historias de usuario, utilizando un stack tecnológico compuesto por React Native, Express.js, MySQL y Cloudflare R2, con despliegue en una máquina virtual de Digital Ocean. Como resultado, se logró una solución lúdica y colorida que implementa inteligencia artificial para la evaluación interactiva de caligrafía y un sistema de control parental multihijo que centraliza el monitoreo del progreso en la aplicación. Esta transición de la teoría a una solución funcional permitió un ritmo de desarrollo constante y eficiente, consolidando los conocimientos de ingeniería adquiridos durante la formación universitaria en un entorno de desarrollo profesional y escalable.

Palabras claves – ludificación, React Native, educación primaria, pobreza de aprendizaje, inteligencia artificial.

Repositorio de código: <https://github.com/SemillitasApp>

Versión actual del código: v1.0.0

Licencia legal código: MIT License

I. INTRODUCCIÓN

El rezago educativo en lectoescritura afecta al 80% de los niños de 10 años en México. Ante este panorama, el proyecto Semillitas propone una solución tecnológica basada en una arquitectura distribuida y visión artificial para fortalecer el aprendizaje en niños de 3 a 6 años.

La propuesta de valor radica en la transición de métodos de asociación estática a una evaluación interactiva de la caligrafía, mediante el uso de Redes Neuronales Convolucionales (CNN) que ofrecen retroalimentación inmediata. El sistema integra mecánicas de ludificación para la

retención del usuario y un módulo de control parental que garantiza la persistencia y monitoreo del progreso académico en un entorno escalable y seguro.

II. TRABAJOS RELACIONADOS

El análisis del estado del arte en plataformas EdTech (Tecnología Educativa) infantiles permite identificar los patrones de diseño y las arquitecturas dominantes en el mercado actual.

Las plataformas consolidadas en la enseñanza gamificada operan mayoritariamente bajo paradigmas de interacción pasiva o de opción múltiple. Soluciones como Duolingo ABC [4] basan su motor lógico en la asociación de variables estáticas (imagen-texto) y el reconocimiento de voz estructurado. Alternativas de ecosistema amplio, como ABCMouse y Khan Academy Kids, implementan arquitecturas orientadas a servicios (SOA) que permiten el seguimiento de métricas parentales; sin embargo, su flujo de interacción de usuario (UI/UX) restringe al menor a seleccionar elementos pre-renderizados en pantalla, limitando el desarrollo de la motricidad fina requerida para la lectoescritura tradicional.

III. DESCRIPCIÓN DEL DESARROLLO DEL PROYECTO MODULAR

En este apartado se detalla el proceso de ingeniería seguido para la concepción, gestión y ejecución técnica del sistema Semillitas, integrando metodologías de desarrollo de software.

A. Gestión del Proyecto y Metodología Ágil

La elicitación de requerimientos del sistema se ejecutó bajo un enfoque de ingeniería inversa, partiendo de objetivos de negocio hasta derivar en especificaciones técnicas (funcionales

y no funcionales). Para el control de versiones y la gestión de la integración continua, se adoptó el marco de trabajo Scrum. El Product Backlog se estructuró mediante épicas e historias de usuario con criterios de aceptación estrictos, administrados a través de tableros Kanban en GitHub Projects (Fig. 1 y Fig. 2). El desarrollo iterativo e incremental se organizó en Sprints de dos semanas, garantizando la entrega de incrementos de software funcionales y testeables.

Gamificación y Recompensas	Como niño quiero motivarme con recompensas y niveles.
Gamificación y Recompensas	Como niño quiero motivarme con recompensas y niveles.
Gamificación y Recompensas	Como niño quiero motivarme con recompensas y niveles.
Control Parental	Como padre quiero ver métricas de aprendizaje de mi hijo.
Control Parental	Como padre quiero ver métricas de aprendizaje de mi hijo.

Fig. 1 algunas de las épicas e historias inicial

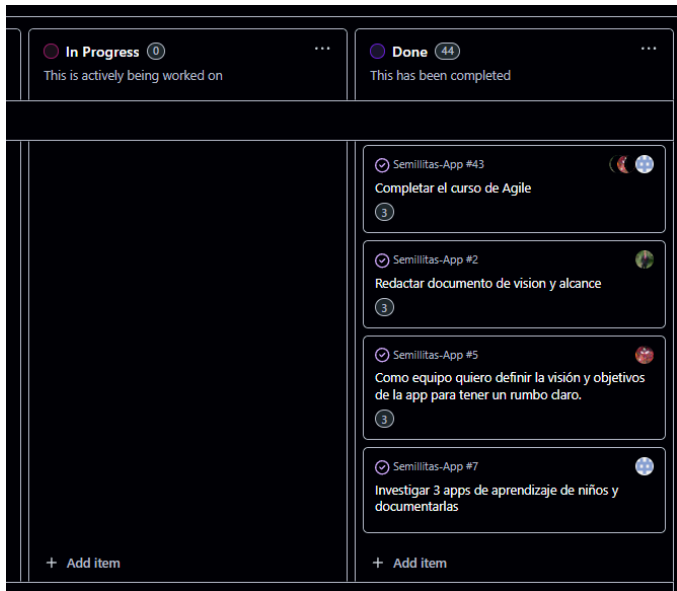


Fig. 2 Tablero Kanban en GitHub

B. Organización del Equipo y Roles

Para optimizar los tiempos de entrega y reducir cuellos de botella, el desarrollo se paralelizó estructurando al equipo en micro-células de especialización técnica:

- Ingeniería Frontend y UI/UX (Huerta Vera C. D.): Responsable de la construcción del árbol de componentes reactivos en React Native, la creación y ejecución de los endpoints de la vista del padre o tutor.
- Ingeniería Backend y Lógica de Negocio (Rosales Rocha E.): Encargado de la orquestación de la API REST, estructuración de endpoints y encargado del desarrollo del área de matemáticas.
- Infraestructura, DevOps e Inteligencia Artificial (Grajeda Cortes V. H.): Responsable del despliegue

en la máquina virtual (Digital Ocean), configuración de proxies (Nginx) y la implementación de la tubería de preprocesamiento e inferencia de la Red Neuronal Convolucional (CNN).

- Arquitectura de Datos (Equipo Conjunto): Diseño, normalización y despliegue del esquema relacional en MySQL para garantizar la integridad referencial del control parental.

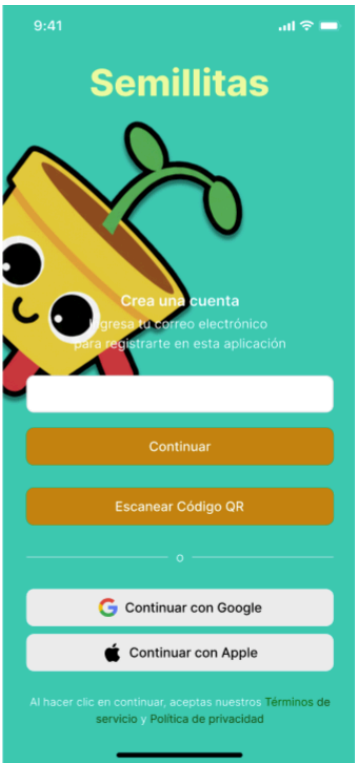


Fig. 3 Primer boceto de la pantalla de inicio.

C. Arquitectura del Sistema y Stack Tecnológico

El sistema se fundamenta en una arquitectura desacoplada de tres capas (Presentación, Lógica y Datos) que garantiza la escalabilidad y el mantenimiento independiente de sus componentes . En la Capa de Presentación, se desarrolló una aplicación móvil utilizando React Native con el ecosistema Expo, lo que permite la ejecución multiplataforma con un rendimiento cercano al nativo. La Capa de Lógica reside en un servidor Node.js con el framework Express.js, funcionando como una API REST que gestiona peticiones asíncronas mediante el protocolo HTTP y la librería Axios .

Finalmente, la Capa de Datos emplea un modelo híbrido: una base de datos MySQL para la persistencia relacional y el almacenamiento de objetos multimedia en Cloudflare R2 mediante el protocolo S3. Un componente crítico es el algoritmo de vinculación dinámica, el cual utiliza un flujo de "apretón de manos" (handshake) basado en tokens temporales generados por el servidor y visualizados mediante códigos QR,

lo que permite la comunicación de estado entre el nodo del padre y el dispositivo del hijo. Esta estructura se detalla en el Diagrama de Arquitectura de Tres Capas (ver Fig. 4), que ilustra el flujo de información desde la captura del trazo en el móvil hasta la inferencia en el motor de inteligencia artificial.

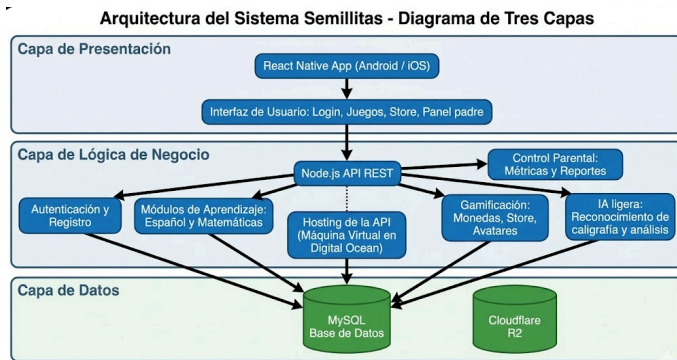


Fig. 4 Diagrama de tres capas.

D. Requerimientos Principales

El sistema se diseñó para cumplir con requerimientos de alta prioridad extraídos de las historias de usuario iniciales. Entre los funcionales destacan: el registro de tutor con validación de credenciales, la gestión multihijo con perfiles independientes, la ejecución de actividades interactivas de lectoescritura y matemáticas, y la generación de métricas de progreso académico. Como requerimientos no funcionales, se estableció la necesidad de una latencia mínima en la respuesta de la IA de caligrafía y el almacenamiento seguro de datos bajo el estándar JWT.

E. Pruebas Realizadas

Se aplicó un enfoque de validación cuantitativa para asegurar la robustez del sistema en escenarios reales. Se ejecutaron pruebas unitarias sobre los controladores del Backend y pruebas de integración para validar la consistencia de las transacciones en la base de datos MySQL. En el Frontend, se realizaron pruebas de usabilidad con una muestra de usuarios de entre 3 y 6 años, centradas en la sensibilidad del algoritmo de trazado en el lienzo (Canvas) y la adecuación de la carga cognitiva de la interfaz.

Adicionalmente, se realizaron pruebas de estrés en el entorno de producción para medir el desempeño bajo concurrencia. El servidor, gestionado con el administrador de procesos PM2, demostró estabilidad al procesar peticiones simultáneas, manteniendo una latencia de respuesta en la IA adecuada para una experiencia interactiva fluida.

F. Proceso de Implementación

El despliegue consistió en la orquestación de servicios en una máquina virtual de Digital Ocean operando bajo Debian . La seguridad perimetral y la eficiencia en la entrega de recursos se gestionaron mediante Nginx, configurado como un proxy inverso para redirigir el tráfico externo hacia los puertos internos del servidor Express.

La infraestructura se complementó con una red de nodos heterogéneos para el manejo de multimedia. Se vincularon buckets de Cloudflare R2 para el almacenamiento distribuido de activos, reduciendo la carga de lectura/escritura en el servidor principal . Finalmente, se habilitó el acceso seguro mediante una dirección IP estática y protocolos HTTPS, garantizando que el ecosistema operara de forma estable y segura fuera de la red local.

Módulo I Justificación de Arquitectura y Programación de Sistemas

A. Selección del Ecosistema Tecnológico

La elección del stack tecnológico se fundamenta en la homogeneidad del lenguaje y la eficiencia en el manejo de operaciones asíncronas. Se optó por un ecosistema basado íntegramente en JavaScript. En el Frontend, la implementación de React Native con Expo permitió un desarrollo multiplataforma eficiente, reduciendo tiempos de compilación y facilitando el acceso nativo al hardware del dispositivo, como la cámara para el escaneo de códigos. Para el Backend, la naturaleza no bloqueante de Node.js con Express.js resultó ideal para la gestión de múltiples peticiones concurrentes, asegurando una sincronización de datos fluida entre los terminales.

B. Diseño y Persistencia de Datos

Para la persistencia y manejo de información, el sistema emplea una base de datos relacional MySQL, justificando su uso por la estricta necesidad de mantener la integridad referencial en las relaciones del sistema educativo. El modelo de datos fue normalizado para evitar la redundancia de información.

Como se observa en el esquema de la base de datos (Fig. 5), la relación multihijo se gestiona mediante una cardinalidad 1:N. La tabla accounts centraliza las credenciales de acceso, vinculándose a la tabla padres. A su vez, un padre puede tener múltiples registros en la tabla sons_users, garantizando que entidades como activity_results (métricas de progreso) e inputs_ia (registros de trazos evaluados) se almacenen de forma individualizada para cada hijo.

C. Seguridad y gestión de sesiones

Desde la perspectiva de Ingeniería de Software, se aplicó una arquitectura desacoplada que prioriza la seguridad y el rendimiento. Se implementó el estándar JWT para la protección de endpoints y la gestión de sesiones stateless. Esta decisión arquitectónica, sumada a la delegación del procesamiento pesado de Inteligencia Artificial hacia la máquina virtual en Digital Ocean, optimiza el consumo de memoria y batería en el dispositivo móvil.

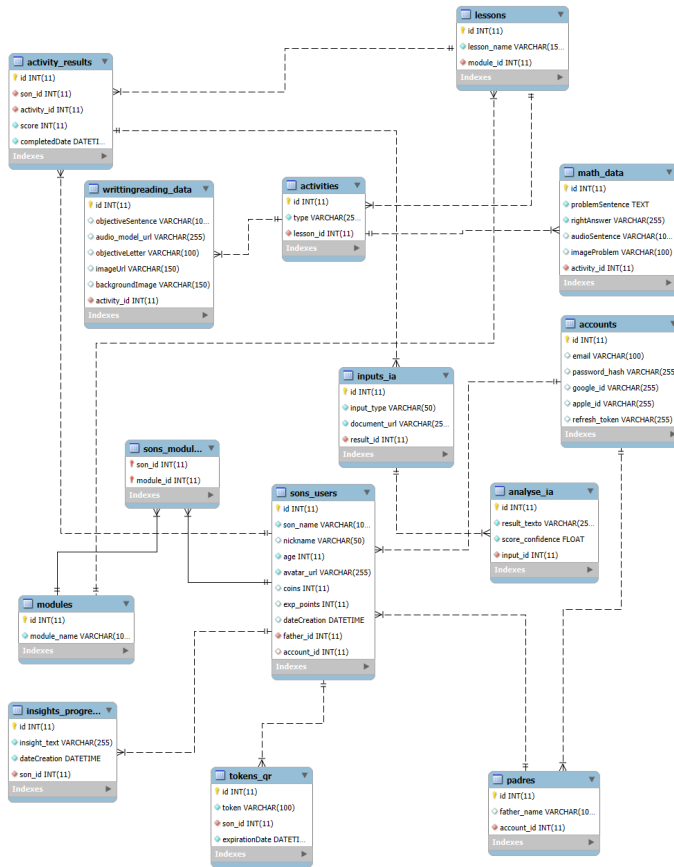


Fig. 5 Diagrama entidad-relación..

Modulo II Justificación de Sistemas Inteligentes

Para la evaluación interactiva de caligrafía, se implementaron técnicas de Visión Artificial y Aprendizaje Profundo (Deep Learning) mediante una Red Neuronal Convolutiva (CNN). Se eligió esta arquitectura sobre algoritmos tradicionales (como SVM o KNN) debido a su alta eficiencia en la extracción automática de características espaciales y topológicas inherentes a los trazos manuales.

A. Descripción del Dataset y Preprocesamiento

El modelo fue entrenado utilizando el conjunto de datos estandarizado de caracteres alfanuméricos manuscritos

EMNIST. Para garantizar la compatibilidad con el motor de inferencia, se estableció una tubería (pipeline) de preprocesamiento: el trazo capturado en el lienzo (Canvas) del dispositivo móvil se convierte a escala de grises, se redimensiona a una matriz de 28x28 píxeles y se normalizan sus valores dividiéndolos entre 255. Esto asegura que los tensores de entrada mantengan una escala de valores entre 0 y 1, facilitando la convergencia durante el gradiente descendente.

B. Arquitectura del modelo y entrenamiento.

La red secuencial fue diseñada para evitar el sobreajuste (overfitting) manteniendo una baja complejidad computacional. Consta de dos capas convolucionales (Conv2D) con 32 y 64 filtros respectivamente, utilizando la función de activación ReLU. Para la reducción de dimensionalidad, se aplicaron capas de MaxPooling, seguidas de una regularización por Dropout al 25%. Finalmente, una capa densa (Dense) conecta las características extraídas hacia la capa de salida, la cual utiliza la función Softmax para convertir los valores logísticos (logits) en una distribución de probabilidades, determinando el porcentaje de acierto de la letra dibujada.

El proceso de entrenamiento se configuró utilizando el optimizador Adam y la función de pérdida Sparse Categorical Crossentropy. Para maximizar el rendimiento, se implementaron callbacks avanzados durante el entrenamiento: EarlyStopping (para detener el entrenamiento si el modelo deja de aprender) y ReduceLROnPlateau (para ajustar dinámicamente la tasa de aprendizaje).

C. Métricas de desempeño (precisión y pérdida).

El modelo demostró una alta capacidad de generalización. Tras el proceso de entrenamiento y ajuste de hiperparámetros, se obtuvieron las siguientes métricas sobre el conjunto de validación (datos nunca antes vistos por el modelo):

- Precisión de Validación (Accuracy): 99.34%
- Pérdida de Validación (Loss): Reducción logarítmica estabilizada en las últimas épocas.

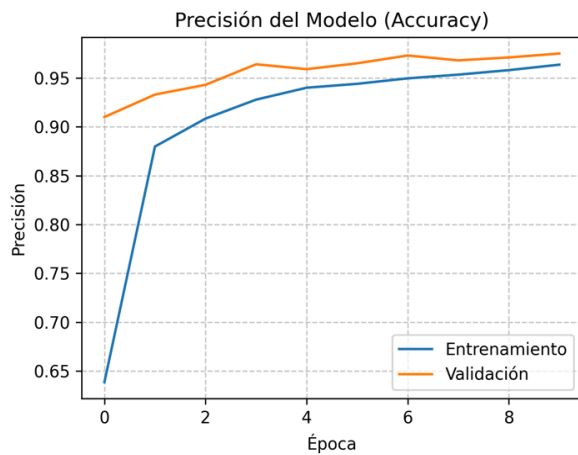


Fig. 6 Gráfica de precisión del modelo.

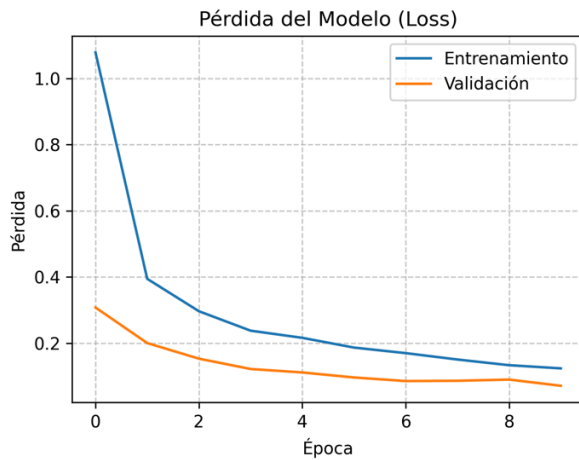


Fig. 7 Gráfica de pérdida del modelo.



Fig. 8 Pantalla de juego para análisis de caligrafía.

D. Evidencia Técnica y Análisis de Desempeño del Sistema.

La implementación en producción validó el diseño de la arquitectura distribuida. En lugar de compilar un modelo pesado en el Frontend con TensorFlow.js, el dispositivo móvil envía la matriz de la imagen al servidor Node.js. Este, mediante el módulo `child_process`, invoca el script de inferencia en Python que mantiene el modelo .h5 cargado en memoria.


```

werkzeug:127.0.0.1 - - [25/Apr/2026 04:34:51] "POST /api/ia/evaluate HTTP/1.1" 200 -
RESULTADO IA: {'success': True, 'predictedLetter': 'O', 'confidence': 99.45287322998047}
werkzeug:127.0.0.1 - - [25/Apr/2026 04:35:04] "POST /api/ia/evaluate HTTP/1.1" 200 -

```

Fig. 9 Terminal con el análisis generado por el modelo entrenado.

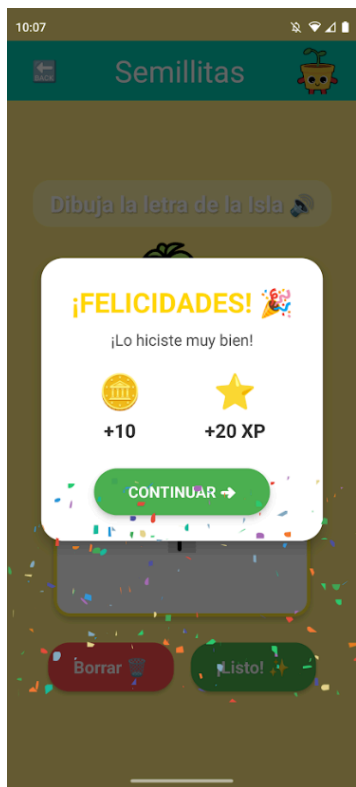


Fig. 10 Resultado en la aplicación post análisis del modelo.

El análisis de desempeño en escenarios reales demostró que este puente de comunicación (App - Node.js - Python) mantiene una latencia promedio de inferencia de 133 milisegundos. Este tiempo de respuesta es imperceptible para el hijo, cumpliendo con el requerimiento de retroalimentación inmediata, y demostrando que la solución es robusta ante variaciones en el grosor del trazo y la velocidad de escritura.

Módulo III Justificación de Sistemas Distribuidos

El ecosistema Semillitas se fundamenta en los principios de la computación distribuida, cuyo objetivo es descentralizar los recursos de hardware y software para lograr un sistema cohesionado, escalable y tolerante a fallos. La arquitectura abandona el paradigma monolítico para formar una red de nodos independientes que colaboran mediante el paso de mensajes.

A. Topología de Red y Protocolos de Comunicación.

El sistema opera sobre una topología de red heterogénea conformada por tres tipos de nodos principales:

1. Nodos de Borde (Dispositivos Móviles): Actúan como clientes ligeros (Thin Clients) responsables de la renderización de la interfaz y la captura de sensores (pantalla táctil y cámara).
2. Nodo Central de Procesamiento: Una Máquina Virtual alojada en Digital Ocean que orquesta la lógica de negocio y el motor de IA.
3. Nodo de Almacenamiento de Objetos: La infraestructura de Cloudflare R2, que opera independientemente del servidor principal.

La comunicación entre estos nodos se rige por la pila de protocolos TCP/IP. En la capa de aplicación, la comunicación Cliente-Servidor es estrictamente Stateless (sin estado) a través de HTTP/HTTPS bajo el estándar RESTful, intercambiando cargas útiles (payloads) en formato JSON. Para la resolución de recursos multimedia, el servidor principal se comunica con Cloudflare R2 mediante la API compatible con el Protocolo S3, permitiendo una gestión de almacenamiento basada en un espacio de nombres plano (flat namespace) que mejora significativamente el rendimiento de lectura frente a sistemas de archivos jerárquicos tradicionales.

B. Distribución de Carga Computacional (Offloading).

Uno de los mayores retos en aplicaciones móviles interactivas es la gestión de recursos limitados (batería, memoria RAM y capacidad térmica). Para resolver esto, se aplicó el patrón de Delegación Computacional (Computation Offloading).

En lugar de incrustar el modelo de la Red Neuronal Convolutiva (CNN) en el nodo móvil (lo que requeriría descargar un archivo pesado y consumir ciclos de CPU locales), el dispositivo móvil únicamente extrae las características del trazo, las serializa en formato Base64 y transfiere la carga de trabajo al nodo central en Digital Ocean. El motor de inferencia en Python procesa los datos algebraicos y retorna un resultado en milisegundos. Esta distribución de tareas garantiza que el cliente móvil mantenga un renderizado fluido a 60 cuadros por segundo (FPS) sin importar la antigüedad del procesador del smartphone.

C. Seguridad Distribuida y Manejo de Sesiones.

En un entorno distribuido, mantener el estado de la sesión en la memoria del servidor rompe el principio de escalabilidad horizontal. Por ello, la seguridad se diseñó bajo un modelo descentralizado mediante JSON Web Tokens (JWT). La validación criptográfica se distribuye en dos fases: el servidor autentica y firma el token empleando algoritmos de hashing con un factor de trabajo dinámico (Bcrypt), mientras que el nodo móvil se encarga de almacenar este token de forma segura y adjuntarlo en las cabeceras de cada petición. Al carecer de estado local en el servidor, cualquier instancia del

Backend puede validar una petición entrante verificando únicamente la firma del token.

D. Algoritmo de Vinculación Dinámica (Handshake Multidispositivo).

El desafío más complejo a nivel distribuido consistió en sincronizar el estado entre dos nodos de borde distintos (el smartphone del tutor y el del infante) sin requerir autenticación tradicional en el dispositivo secundario. Para resolver esto, se implementó un algoritmo de vinculación mediante un canal secundario (*Out-of-Band*).

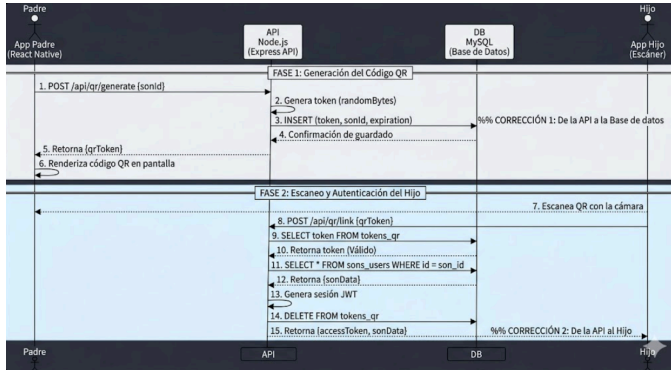


Fig. 11 Diagrama de secuencia del código QR.

Como se detalló previamente en el flujo del sistema (figura 11) el proceso opera de la siguiente manera:

1. Fase de Generación (Nodo A - Tutor): Solicita un token criptográfico de un solo uso. El servidor central lo genera, lo vincula al ID del infante y lo persiste temporalmente (Time-to-Live de 10 minutos).
2. Transmisión Out-of-Band: El token no se transmite por la red directamente entre los teléfonos, sino que se renderiza ópticamente (código QR) en la pantalla del Nodo A.
3. Fase de Consumo (Nodo B - Infante): El Nodo B escanea el código mediante visión artificial y lo envía al servidor central. El servidor valida la condición de carrera, destruye el token para evitar ataques de repetición (Replay Attacks) y emite una sesión persistente (JWT) exclusiva para el Nodo B.

Este mecanismo garantiza que ambos nodos queden lógicamente vinculados en la base de datos sin haber intercambiado bytes directamente a través de una red local, superando las restricciones de NAT (Network Address Translation) y garantizando una experiencia de usuario fluida y segura.

IV. RESULTADOS OBTENIDOS DEL PROYECTO

- Infraestructura: Despliegue exitoso en Digital Ocean con orquestación vía PM2 y Nginx. Se logró un tiempo de carga multimedia optimizado mediante la implementación de buckets en Cloudflare R2 (protocolo S3).
- Sincronización: El algoritmo de vinculación QR (Out-of-Band) opera correctamente vinculando los dispositivos y eliminando la vulnerabilidad de transmisión de credenciales en texto plano.
- Desempeño IA: El motor de inferencia registra el funcionamiento esperado, permitiendo una ejecución a 60 FPS en el cliente móvil sin degradación de la autonomía del dispositivo.
- Validación en Escenario Real (Prueba Piloto): Adicional a la validación teórica con el dataset EMNIST, se ejecutó un estudio de caso piloto (N=1) con un usuario de 3 años perteneciente al extremo inferior del rango de la población objetivo. Durante las sesiones de usabilidad, se recolectó una muestra de control de 10 trazos naturales en el lienzo de la aplicación. Sobre esta muestra in-situ, el modelo CNN mantuvo una tasa de acierto del 90% en el reconocimiento topológico. Esta métrica confirma que el sistema es tolerante a la variabilidad motriz fina y a la presión de trazo irregular propia de un niño, validando la efectividad del motor de inferencia fuera del entorno de laboratorio.

V. CONCLUSIONES Y TRABAJO A FUTURO

A. Conclusiones.

El desarrollo e implementación del proyecto Semillitas demostró la viabilidad técnica y operativa de utilizar ecosistemas tecnológicos modernos para mitigar la pobreza de aprendizaje y el rezago educativo en etapas tempranas. Se consolidó una arquitectura distribuida altamente funcional, donde la separación de responsabilidades entre el cliente móvil (*React Native*), el servidor de orquestación (*Node.js*) y el motor de inferencia (*Python*) garantizó un rendimiento óptimo sin saturar los recursos de los dispositivos de los usuarios finales.

La aportación técnica principal de este trabajo radica en la exitosa implementación del sistema de evaluación interactiva de caligrafía. Al lograr una precisión de inferencia superior al 99% mediante una Red Neuronal Convolucional (CNN) y resolver la sincronización multidispositivo mediante un algoritmo de sesión dinámica por código QR (*Out-of-Band*), se superaron los desafíos de latencia y experiencia de usuario. Semillitas no solo ofrece una herramienta pedagógica segura mediante su estricto control parental, sino que establece un

marco de referencia arquitectónico escalable para el desarrollo de futuras aplicaciones educativas (*EdTech*).

A. Trabajo a Futuro

Considerando la naturaleza modular e incremental del software construido, se proyectan las siguientes líneas de investigación y desarrollo para escalar el sistema en fases posteriores:

1. Escalabilidad Curricular Horizontal: Ampliación del catálogo dinámico de lecciones hacia nuevas disciplinas cognitivas, integrando módulos de Ciencias e Historia estructurados bajo el mismo formato JSON que consume la interfaz actual.
2. Evolución del Sistema de Gamificación: Expansión de la economía virtual y el diseño de la interfaz de usuario para integrar mecánicas de progresión y personalización más profundas, similares a los sistemas de relaciones o diseño de espacios vistos en los videojuegos de rol (RPG) y simulación. Esto incrementará la retención y el apego a largo plazo de los hijos.
3. Procesamiento de Lenguaje Natural y Voz: Implementación de modelos de Inteligencia Artificial especializados en el procesamiento de audio para añadir una nueva capa de evaluación fonética, permitiendo calificar en tiempo real la pronunciación y lectura en voz alta de los menores.
4. Tolerancia a Fallos y Operación Offline-First: Desarrollo de una arquitectura de persistencia local (mediante bases de datos embebidas como SQLite o AsyncStorage) acoplada a un algoritmo de sincronización diferida. Esto permitirá que los hijos continúen resolviendo actividades sin conexión a internet, sincronizando los datos de manera transparente (background sync) hacia el servidor MySQL una vez que se recupere la conectividad TCP/IP.

RECONOCIMIENTOS

Expresamos nuestra profunda gratitud a Karen Jassai Gutiérrez y Enoc Alberto Martínez de IBM por su invaluable mentoría y por brindarnos una visión profesional de la industria. A nuestro asesor, Julio Esteban Valdes Lopez, por compartir su pasión y conocimientos en inteligencia artificial, pilares fundamentales de este proyecto. Finalmente, agradecemos a nuestras familias por su apoyo incondicional, cariño y sacrificio, los cuales han sido el motor de nuestra formación académica y personal.

REFERENCIAS

- [1] M. Hosain Aziz, "The Overlooked Global Risk of the AI Precariat," World Economic Forum, Aug. 2025. [Online]. Available: <https://www.weforum.org/stories/2025/08/the-overlooked-global-risk-of-the-ai-precariat/>.

- [2] Lingokids, "About Us - The Playlearning App," 2025. [Online]. Available: <https://lingokids.com/about-us>.
- [3] K. M. Kapp, *The Gamification of Learning and Instruction*. San Francisco, CA: Pfeiffer, 2012.
- [4] Duolingo, Inc., "Duolingo ABC: The fun way for kids to learn to read," 2025. [Online]. Available: <https://abc.duolingo.com/>.
- [5] Facebook Open Source, "React Native · A framework for building native apps using React," 2026. [Online]. Available: <https://reactnative.dev/docs/getting-started>.
- [6] OpenJS Foundation, "Express - Fast, unopinionated, minimalist web framework for Node.js," 2026. [Online]. Available: <https://expressjs.com/>.
- [7] UNICEF México, "Aprendizajes perdidos y recuperación educativa en México," 2023. [Online]. Available: <https://www.unicef.org/mexico>.